

8)

1) Which optimization produced the greatest improvement?

⇒ The single-pass broadcast with inline dead-client cleanup made the biggest practical difference. Previously, a dead socket left in the clients dictionary would cause `send()` failures on every subsequent broadcast until it was manually removed. Detecting and removing dead clients within the same lock protected loop that sends the message meant the server stayed stable through all three test runs (5, 8 and 10 concurrent clients) with no crashes which was the single biggest requirement of this assignment.

2) Why is scalability important?

⇒ An application must handle real-world user loads not just a few test clients. Poor scalability means crashes, lag, or wasted resources as usage grows making the app unreliable in production.

3) What reliability issues did you discover?

⇒ Dead sockets weren't detected until the next broadcast attempt, briefly causing failed sends. Also, silently dropped clients (no /logout) keep occupying server threads until session-timeout logic was added.

4) Which optimization was the most difficult?

⇒ Getting automatic reconnection - the client had to tell apart a real network failure (reconnect), a server-sent ~~logic~~ logout (don't reconnect), and a manual logout (don't reconnect), since mixing these up caused unwanted re-login prompts.

5. What additional improvements are required to support 400 users?

⇒ Replace the single lock protected sequential broadcast with per-client queues or an async I/O model (eg: asyncio) so one slow client can't delay everyone else, and consider a pub-sub backend for horizontal scaling.